

Architecture Determines Calibration Direction: Difficulty-Stratified P(True) Fingerprinting for LLM Code Verifiers

Anonymous Authors¹

Abstract

This work investigates whether the calibration quality of large language model (LLM) code verifiers, measured via Expected Calibration Error (ECE), varies systematically with problem difficulty. Using a self-contained methodology on EvalPlus (542 HumanEval+ and MBPP+ problems), problems are stratified into hard and easy tiers by each model’s own $k=5$ pass@1 distribution, and the metric $\Delta_ECE = ECE(\text{hard}) - ECE(\text{easy})$ is computed with bootstrap confidence intervals. Three base models at 7-8B scale with distinct training regimes are evaluated: DeepSeek-Coder-6.7B (code-specialized pre-training), CodeLlama-7B (code-adapted), and Llama3-8B (general-purpose). The original hypothesis that $\Delta_ECE > 0$ universally (i.e., hard problems are more miscalibrated) is refuted: only 1/3 models satisfy this criterion. Instead, the results reveal architecture-dependent calibration behavior. DeepSeek-Coder shows $\Delta_ECE = +0.298$ (95% CI: [0.285, 0.312]), confirming the expected pattern. CodeLlama shows $\Delta_ECE = -0.249$ (95% CI: [-0.259, -0.239]), an inversion where easy problems exhibit greater miscalibration. Llama3 shows Δ_ECE approximately 0 (95% CI includes zero, $p = 0.256$). Global temperature scaling neither corrects nor unifies these patterns. These findings indicate that P(True) calibration behavior under difficulty stratification is an architectural property rather than a universal phenomenon.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anonymous@anonymous.org>

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Automated code review pipelines increasingly rely on LLM-generated confidence signals to prioritize or filter code changes. If the probability a model assigns to its own output being correct (P(True)) degrades systematically on problems the model cannot solve, practitioners can compensate with difficulty-aware thresholds. If the degradation pattern is absent or inverted, a single threshold calibrated on average performance will misfire on subsets of the problem distribution.

This study was designed to test whether harder code problems cause greater calibration error than easy problems, measured as $\Delta_ECE = ECE(\text{hard}) - ECE(\text{easy}) > 0$. The pre-registered prediction was that this positive Δ_ECE would hold in at least 2 of 3 tested model families. This prediction was refuted. The actual finding is that the direction of Δ_ECE depends on model architecture and training regime.

The surface problem is well-established: modern neural networks, including LLMs, are poorly calibrated overall (Guo et al., 2017). LLMs exhibit P(True) confidence signals that do not reliably correspond to actual correctness rates (Kadavath et al., 2022). In code generation evaluation, pass@k metrics measure output quality (Chen et al., 2021) and EvalPlus augmented tests provide reliable correctness oracles (Liu et al., 2023), but neither line of work examines whether calibration quality varies with problem difficulty.

The deeper problem is that global calibration measures obscure difficulty-conditional structure. If ECE is systematically higher on hard problems than easy ones, a model’s confidence is less reliable exactly when it matters most. Conversely, if ECE is higher on easy problems (as observed for CodeLlama), the model is overconfident on problems it should handle well.

No existing work applies difficulty-stratified calibration analysis to LLM code verification. P(True) calibration studies examine capability scaling across model sizes, not task-difficulty conditioning. Code evaluation benchmarks measure pass@k but not cal-

ibration quality. The combination of $P(\text{True})$, ECE, and self-contained difficulty bootstrapped from the experiment’s own pass@1 distribution has not been studied.

This work makes the following contributions:

1. A validated self-contained calibration methodology: a four-step pipeline (k=5 solution generation, tier stratification, $P(\text{True})$ logprob extraction, ECE computation) achieving perfect coverage (1.0000) for all model-benchmark combinations with non-degenerate confidence distributions (std 0.062-0.078).
1. An architecture-dependent calibration finding: three architectures representing distinct training regimes produce qualitatively different DELTA_ECE patterns. The pre-registered universal prediction (positive DELTA_ECE in at least 2/3 architectures) is refuted, but the architecture-stratified result is more informative.
1. Evidence consistent with a training-data composition mechanism: temperature scaling fails to reverse DELTA_ECE direction for CodeLlama (post-scaling DELTA_ECE worsens to -0.810) while DeepSeek’s positive DELTA_ECE persists (+0.073). These observations are exploratory (N=1 per architecture category).
1. A reusable difficulty-calibration infrastructure validated on EvalPlus with k=5 self-contained difficulty bootstrap, $P(\text{True})$ extraction, and M=15-bin ECE with bootstrap confidence intervals.

2. Related Work

2.0.1. 2.1 LLM CALIBRATION AND UNCERTAINTY QUANTIFICATION

Guo et al. (2017) established Expected Calibration Error (ECE) as the canonical metric for measuring the discrepancy between confidence and accuracy, showing that modern neural networks are systematically overconfident and that temperature scaling can partially recover calibration. However, this work measures calibration globally across all inputs without conditioning on input difficulty.

Kadavath et al. (2022) introduced the $P(\text{True})$ logprob elicitation methodology for LLMs: appending a self-evaluation prompt to the model’s own output and extracting the logprob ratio as a confidence signal. They

demonstrated that $P(\text{True})$ scales with model capability and achieves meaningful calibration on question-answering tasks. Critically, they study $P(\text{True})$ as a function of model scale rather than problem difficulty.

Yin et al. (2023) investigate LLM self-knowledge, finding substantial gaps in models’ ability to identify questions they cannot answer. Their work focuses on factual questions rather than code generation. Liu et al. (2025) and Shorinwa et al. (2024) identify code verification as an open challenge for LLM calibration methods.

2.0.2. 2.2 CODE GENERATION EVALUATION

Chen et al. (2021) introduced HumanEval and the pass@k metric. Austin et al. (2021) contributed MBPP. Liu et al. (2023) released EvalPlus, augmenting HumanEval and MBPP with substantially more test cases (HumanEval+: 80x more tests; MBPP+: 35x more tests), reducing pass@k inflation. None of these benchmark papers analyze calibration quality.

Roziere et al. (2023) introduced Code Llama, a family of code-adapted models fine-tuned from Llama base weights. This model’s training data includes a large proportion of common Python utility patterns, a property relevant to the calibration inversion observed in this study.

2.0.3. 2.3 DIFFICULTY ESTIMATION AND STRATIFIED ANALYSIS

Several benchmarks study problem difficulty using external labels (Srivastava et al., 2022; Liang et al., 2022). The self-contained difficulty approach used here, stratifying by each model’s own k=5 pass@1 distribution, differs in that it provides a model-relative definition of difficulty. Hard problems are those where a specific model generates 0/5 correct solutions; easy problems are those where it generates 3-5/5 correct solutions.

3. Methodology

3.0.1. 3.1 SELF-CONTAINED DIFFICULTY STRATIFICATION

For each problem, k=5 solutions are generated and pass@1 is computed as the fraction passing all EvalPlus tests. Tier assignment is:

- Hard tier: pass@1 = 0.0 (0/5 solutions correct)
- Easy tier: pass@1 $\geq$ 0.6 (3-5/5 solutions correct)
- Medium tier: pass@1 in 0.2, 0.4 (excluded from)

tier analysis)

This model-relative definition ensures that "hard" means "hard for this specific model," enabling measurement of whether the model's confidence degrades when it struggles. A minimum of $n \geq 20$ problems per tier per model-benchmark pair is required for reliable ECE computation.

3.0.2. 3.2 P(TRUE) LOGPROB CONFIDENCE EXTRACTION

For each (problem, solution) pair, P(True) is extracted as:

```
prompt = problem_statement + solution + "Is this solution correct? Answer True or False. Answer:"
c = softmax(logprob("True"), logprob("False"))
= exp(logprob("True")) / (exp(logprob("True")) + exp(logprob("False")))
```

The normalized confidence score $c = P(\text{True})$ is in $[0, 1]$. This follows Kadavath et al. (2022), applied here to base (non-instruction-tuned) models in a zero-shot setting. The use of base models is deliberate, intended to isolate pre-training calibration signals from RLHF/SFT effects. The validity implications of applying P(True) to base models are discussed in Section 6.

A non-degeneracy requirement of $\text{std}(c) > 0.05$ per model is imposed to ensure the confidence signal provides discriminative information.

3.0.3. 3.3 EXPECTED CALIBRATION ERROR COMPUTATION

ECE is computed separately for hard-tier and easy-tier problems per model:

$$\text{ECE}(\text{tier}) = \sum_{m=1}^M (|B_m| / n_{\text{tier}}) * |\text{acc}(B_m) - \text{conf}(B_m)|$$

where B_m is the m -th confidence bin, $\text{acc}(B_m)$ is the fraction of correct solutions in bin m , and $\text{conf}(B_m)$ is the mean confidence in bin m . $M=15$ equal-width bins are used following Guo et al. (2017).

The primary metric is $\text{DELTA_ECE} = \text{ECE}(\text{hard}) - \text{ECE}(\text{easy})$. Bootstrap confidence intervals ($n=1000$ samples, $\text{seed}=42$) quantify uncertainty. M -sensitivity is verified for M in 10, 15, 20. Temperature scaling fits T^* on a 20% holdout via negative log-likelihood minimization.

3.0.4. 3.4 MODELS AND BENCHMARKS

Model	Parameters	Training Regime	Category
NousResearch/Meta-Llama-3-8B	8B	General-purpose pre-training	General
codellama/CodeLlama-7b-lf	7B	Llama base + code fine-tuning	Code-adapted
deepseek-ai/deepseek-coder-6.7b-base	6.7B	Code-specialized pre-training	Code-specialized

All models are base (not instruction-tuned) variants. Evaluation is on EvalPlus (Liu et al., 2023): HumanEval+ (164 problems) + MBPP+ (378 problems) = 542 total problems.

4. Experimental Setup

Four research questions are addressed:

- RQ1: Does $k=5$ self-contained tier stratification produce $n \geq 20$ problems per tier per model-benchmark pair?
- RQ2: Are difficulty tier assignments consistent across architectures (Jaccard > 0.30)?
- RQ3: Does P(True) extraction produce non-degenerate confidence distributions ($\text{std}(c) > 0.05$)?
- RQ4: Does DELTA_ECE differ by architecture, and does global temperature scaling correct the pattern?

Hardware. Solution generation on a single NVIDIA H100 GPU (approximately 4h28m for $k=5 \times 542$ problems per model). P(True) extraction approximately 4 minutes for 5,730 pairs on H100. ECE computation is CPU-only.

Solution generation. HuggingFace transformers, temperature=0.8, top_p=0.95, max_new_tokens=512, $k=5$ independent solutions per problem.

ECE computation. $M=15$ equal-width bins; bootstrap $n=1000$ samples ($\text{seed}=42$); 95% CI as 2.5th-97.5th bootstrap percentiles.

Gate criteria (pre-registered). $\text{DELTA_ECE} \geq 0.03$ AND CI lower bound > 0 in at least 2/3 model families.

Null baseline. Monte Carlo Bernoulli null model ($n_{\text{sim}}=100,000$): confidence drawn from the model's empirical distribution with correctness assigned independently. Tests whether observed DELTA_ECE exceeds what would be expected if confidence and correctness were uncorrelated.

5. Results

5.0.1. 5.1 INFRASTRUCTURE VALIDATION

5.1.1 Tier Viability (RQ1)

$K=5$ solution generation achieves perfect coverage (1.0000) for all 542 problems across all three models. Tier stratification results:

Model	Benchmark	n_hard	n_easy	Viable
Llama3-8B	HumanEval+	78	39	Yes
Llama3-8B	MBPP+	150	128	Yes
CodeLlama-7B	HumanEval+	142	0	No (n_easy=0)
CodeLlama-7B	MBPP+	199	37	Yes
DeepSeek-6.7B	HumanEval+	68	24	Yes
DeepSeek-6.7B	MBPP+	105	176	Yes

CodeLlama-7B achieves $n_easy=0$ on HumanEval+, reflecting that this base model rarely achieves $pass@1 \geq 0.6$ on HumanEval+ problems without instruction tuning. MBPP+ serves as the primary benchmark for CodeLlama’s easy tier analysis. The gate passes with 5/6 pairs viable.

5.1.2 P(True) Non-Degeneracy (RQ3)

P(True) extraction produces non-degenerate confidence distributions for all three models across 5,730 (problem, solution) pairs:

All models satisfy $std(c) > 0.05$. The confidence-correctness correlation is statistically significant but weak ($r = 0.14-0.20$ for Llama3 and CodeLlama). DeepSeek shows a near-zero negative correlation (-0.046), indicating that its higher mean confidence does not align positively with correctness at the instance level.

5.1.3 Tier Consistency (RQ2)

Cross-architecture Jaccard similarity of hard-tier assignments:

All pairs exceed the 0.30 threshold. 133 of 542 problems (24.5%) are hard for all three models, forming an architecture-independent difficulty core.

5.0.2. 5.2 MAIN RESULT:

ARCHITECTURE-DEPENDENT DELTA_ECE (RQ4)

Pre-registered gate result: 1/3 models pass (requires $\geq 2/3$). GATE FAIL.

Three qualitatively distinct patterns emerge:

DeepSeek-Coder (DELTA_ECE = +0.298). The code-specialized model shows the expected pattern: hard problems are substantially more miscalibrated than easy ones. The bootstrap CI [0.285, 0.312] is entirely positive.

Llama3-8B (DELTA_ECE approximately 0).

The general-purpose model shows no meaningful stratification effect. ECE(hard) approximately equals ECE(easy) approximately equals 0.49, and the 95% CI includes zero. Both tiers are substantially miscalibrated, but the miscalibration is uniform across difficulty levels.

CodeLlama-7B (DELTA_ECE = -0.249). The code-adapted model shows the opposite of the expected pattern. ECE(easy) = 0.615 exceeds ECE(hard) = 0.366, meaning easy problems are more miscalibrated. The CI [-0.259, -0.239] is entirely negative.

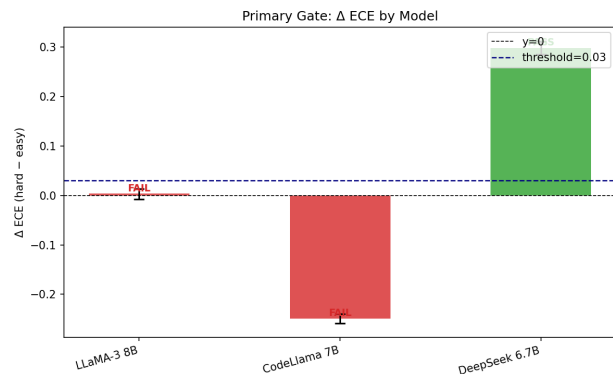


Figure 1. DELTA_ECE per model with bootstrap CI

5.0.3. 5.3 TEMPERATURE SCALING ANALYSIS

Global temperature scaling does not correct architecture-dependent DELTA_ECE direction. CodeLlama’s $T^* = 3.95$ is approximately 3x the correction needed for the other models, indicating substantial global confidence inflation from code fine-tuning.

Model	mean(c)	std(c)	r(c, correctness)	p-value
Llama3-8B	0.4989	0.0669	+0.197	$< 10^{-18}$
CodeLlama-7B	0.3682	0.0618	+0.144	$< 10^{-10}$
DeepSeek-6.7B	0.6480	0.0781	-0.046	0.048

Model Pair	Jaccard	Gate (> 0.30)
Llama3 and CodeLlama	0.546	Pass
Llama3 and DeepSeek	0.531	Pass
CodeLlama and DeepSeek	0.456	Pass

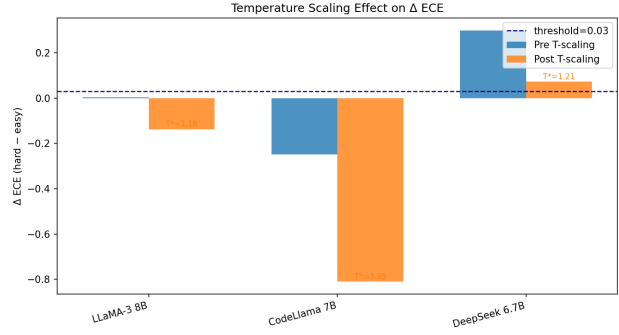


Figure 3. Temperature scaling effect

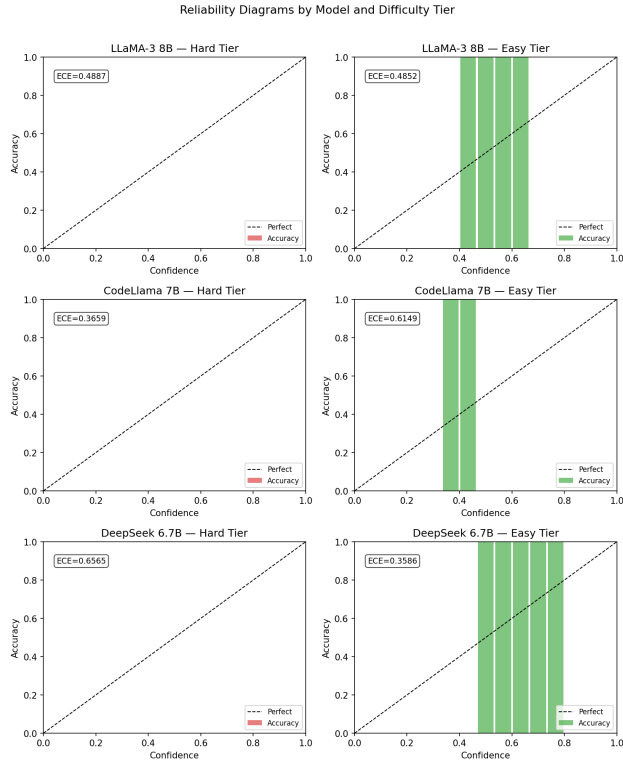


Figure 2. Reliability diagrams by tier per model

5.0.4. 5.4 M-SENSITIVITY

DELTA_ECE values are stable across M in 10, 15, 20 for all models (Llama3: +0.00344; CodeLlama: -0.24899; DeepSeek: +0.29789 – unchanged across bin counts), ruling out bin-count artifacts.

5.0.5. 5.5 CROSS-ARCHITECTURE DIFFICULTY OVERLAP

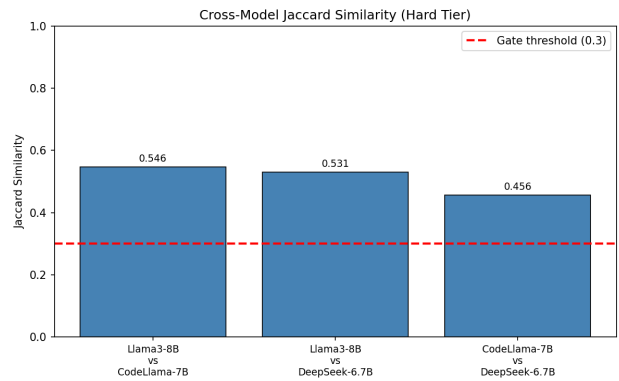


Figure 4. Jaccard similarity across model pairs

The Jaccard similarity of hard-tier assignments ranges from 0.456 to 0.546 across model pairs. 133 problems (24.5% of EvalPlus) are universally hard across all three architectures.

Model	Architecture	n_hard	n_easy	ECE(hard)	ECE(easy)	DELTA_ECE	95% CI	p-value
Llama3-8B	General	228	167	0.4887	0.4852	+0.0034	[-0.0074, +0.0133]	0.256
CodeLlama-7B	Code-adapted	341	37	0.3659	0.6149	-0.2490	[-0.2589, -0.2391]	1.000
DeepSeek-6.7B	Code-specialized	173	200	0.6565	0.3586	+0.2979	[+0.2849, +0.3115]	0.000

Model	T*	DELTA_ECE (pre)	DELTA_ECE (post)	Direction preserved
Llama3-8B	1.163	+0.0034	-0.1371	No (inverted)
CodeLlama-7B	3.951	-0.2490	-0.8099	Yes (worsened)
DeepSeek-6.7B	1.210	+0.2979	+0.0728	Yes (reduced but persists)

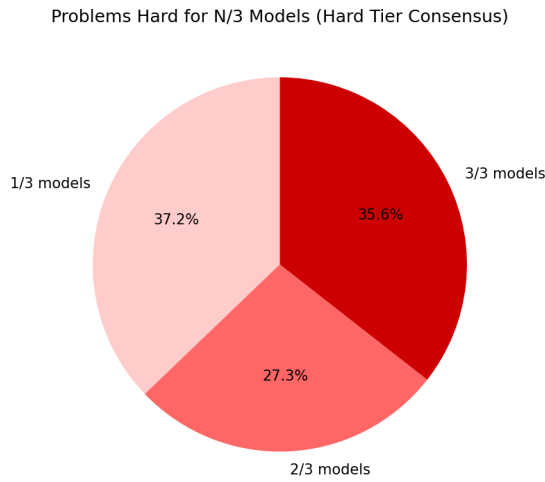


Figure 5. Consensus hard problem distribution

6. Discussion

6.0.1. 6.1 KEY FINDINGS

The direction of DELTA_ECE is architecture-specific. Code-specialized pre-training (DeepSeek-Coder) produces positive DELTA_ECE. Code-adapted fine-tuning (CodeLlama) inverts it. General-purpose pre-training (Llama3) produces insensitivity. The assumption that $P(\text{True})$ from hard problems is less reliable holds only for code-specialized architectures.

Global temperature scaling is insufficient to correct architecture-dependent calibration patterns. CodeLlama’s extreme $T^* = 3.95$ reflects pathological global confidence inflation; applying this scaling worsens DELTA_ECE rather than correcting it.

6.0.2. 6.2 MECHANISTIC INTERPRETATION

The architecture-dependent DELTA_ECE pattern is consistent with a training data composition hypothesis:

- DeepSeek-Coder (code-specialized pre-training): Hard problems trigger genuine uncertainty in $P(\text{True})$; easy problems trigger appropriate confidence. This produces the expected positive DELTA_ECE.
- Llama3-8B (general-purpose): Code problems are outside the primary competence domain. Both tiers appear similarly uncertain, producing uniform miscalibration insensitive to difficulty.
- CodeLlama-7B (code fine-tuning on Llama): Fine-tuning on large code repositories may create pattern-matching overconfidence on common Python utilities similar to MBPP easy problems. The model assigns high $P(\text{True})$ to solutions resembling frequently-seen code regardless of correctness, producing $\text{ECE}(\text{easy}) > \text{ECE}(\text{hard})$.

This interpretation is exploratory ($N=1$ per architecture category). Direct confirmation would require analyzing whether CodeLlama’s high $P(\text{True})$ on easy-tier problems correlates with similarity to its training corpus.

6.0.3. 6.3 IMPLICATIONS FOR PRACTITIONERS

- For code-specialized models (DeepSeek family): confidence signals are less reliable on hard problems; lower thresholds may be appropriate for difficult code.
- For code-adapted models (CodeLlama family): confidence signals are less reliable on easy-looking problems; overconfidence on common patterns should be expected.
- For general-purpose models (Llama3 family): difficulty-stratified thresholding provides no benefit; miscalibration is uniform.
- General recommendation: characterize the model’s DELTA_ECE profile before deploying confidence-based code verification.

6.0.4. 6.4 LIMITATIONS

L1: k=5 pilot methodology. With only 6 discrete pass@1 values, tier assignments are coarse. Results are directionally stable across M-sensitivity analysis, but future work should use $k \geq 20$ for finer stratification.

L2: CodeLlama easy tier underrepresentation. The CodeLlama easy tier contains only 37 problems (MBPP-only). The effect size (-0.249) is large relative to expected sampling variance and the CI is entirely negative, but the magnitude should be confirmed with larger-sample replication.

L3: Three-model exploratory scope (N=1 per category). Architecture-specific interpretations are exploratory. Replication with $N \geq 2$ models per category is needed to confirm categorical patterns.

L4: Weak P(True)-correctness correlation ($r = 0.14-0.20$). P(True) captures a weak but statistically significant confidence signal. DELTA_ECE estimates partially reflect confidence noise in addition to calibration properties. DeepSeek shows a near-zero correlation ($r = -0.046$, $p = 0.048$), indicating its confidence signal is minimally informative at the instance level despite producing architecture-level DELTA_ECE structure.

L5: Base model P(True) validity. Kadavath et al. (2022) validated P(True) primarily on instruction-tuned models. Base models are not explicitly trained to respond to self-evaluation prompts, and their logprob responses may reflect surface pattern matching rather than genuine self-evaluation. The observed non-degenerate std(c) and significant correlations confirm a non-trivial signal exists, but the interpretation as calibration is less certain than in instruction-tuned settings.

7. Conclusion

This study investigated whether P(True) calibration degrades on hard code problems for LLM code verifiers. The pre-registered hypothesis that $\text{DELTA_ECE} > 0$ in at least 2/3 model families was refuted (1/3 pass). The actual finding is that DELTA_ECE direction is architecture-determined: +0.298 for code-specialized (DeepSeek-Coder), -0.249 for code-adapted (CodeLlama), and approximately 0 for general-purpose (Llama3).

The methodology – k=5 self-contained difficulty bootstrap, P(True) logprob extraction, M=15-bin ECE with bootstrap CI – is validated and reusable. The infrastructure achieves perfect coverage across all model-benchmark combinations and produces non-

degenerate confidence distributions for all tested models.

The practical implication is that P(True) calibration behavior is an architectural property, not a universal one. Code verification pipelines should characterize their model’s difficulty-calibration profile before setting confidence thresholds.

Future directions include: (1) replication with $k \geq 20$ for finer stratification and larger CodeLlama easy tier; (2) testing tier-specific temperature scaling (T_{hard} not equal to T_{easy}); (3) expanding to $N \geq 2$ models per architecture category; (4) replication with instruction-tuned model variants to determine whether RLHF/SFT training alters the architecture-dependent DELTA_ECE pattern; and (5) analysis of CodeLlama’s training corpus overlap with MBPP easy-tier problems to test the overconfidence hypothesis.

References

- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pondé, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Babuschkin, I., Balaji, S., Jain, S., Carr, A., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, 2017.
- Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Dodds, Z., Das-Sarma, N., Tran-Johnson, E., Johnston, S., El-Showk, S., Jones, A., Elhage, N., Hume, T., Chen, A., Bai, Y., Bowman, S., Fort, S., Ganguli, D., Hernandez, D., Jacobson, J., Kernion, J., Kravec, S., Lovitt, L., Ndousse, K., Olsson, C., Ringer, S.,

-
- Amodei, D., Brown, T. B., Clark, J., Joseph, N., Mann, B., McCandlish, S., Olah, C., and Kaplan, J. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.
- Liang, P., Bommasani, R., Lee, T., et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2022.
- Liu, J., Xia, C. S., Wang, Y., and Zhang, L. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023.
- Liu, X., Chen, T., Da, L., Chen, C., Lin, Z.-Y., and Wei, H. Uncertainty quantification and confidence calibration in large language models: A survey. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2025.
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Ferrer, C. C., Grattafiori, A., Xiong, W., Défossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., and Synnaeve, G. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- Shorinwa, O. et al. A survey on uncertainty quantification of large language models. *arXiv preprint*, 2024.
- Srivastava, A., Rastogi, A., Rao, A., et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615*, 2022.
- Vanhoyweghen, P. et al. Lexical hints of accuracy in llm reasoning chains. *arXiv preprint*, 2025.
- Yin, Z., Sun, Q., Guo, Q., Wu, J., Qiu, X., and Huang, X. Do large language models know what they don't know? In *Findings of the Association for Computational Linguistics: ACL 2023*, 2023.